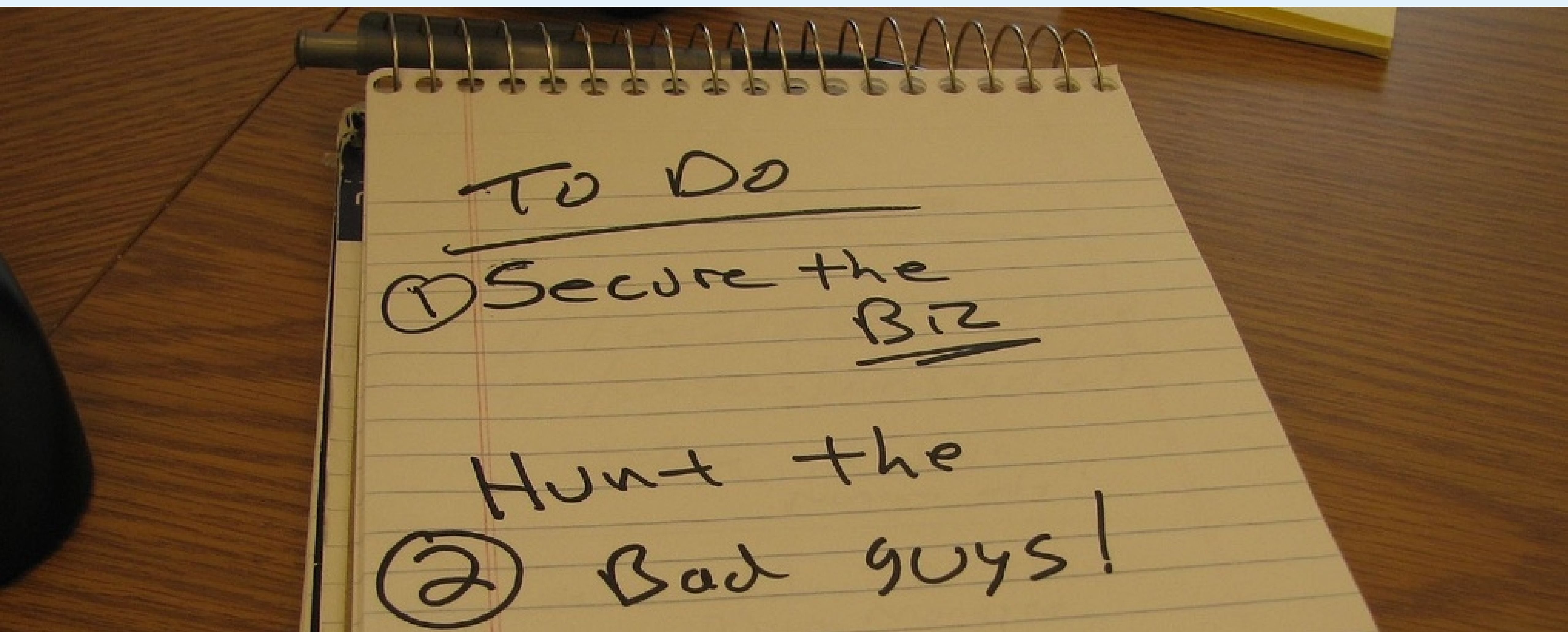


ToDo & Co

AUTHENTIFICATION DE L'APPLICATION

Documentation technique



2026

Sommaire

 Introduction	1
 Vue d'ensemble	2
 Fichiers concernés	3
 Configuration de Symfony Security	4
 Hashage des mots de passe	5
 Fournisseur d'utilisateurs	6
 Entité User	7
 Identifiant de l'utilisateur	8
 Stockage des utilisateurs	9
 Mot de passe	10
 Page de connexion	11
 Formulaire de connexion	12
 Fonctionnement du formulaire	13
 Firewall	14
 Déconnexion	15
 Contrôle des accès	16
 Accès à /login	17
 Gestion des utilisateurs	18

Suite du sommaire

	Accès aux autres pages	19
	Gestion des rôles	20
	Modification du rôle	21
	Association avec les tâches	22
	Autorisation des actions sur les tâches	23
	Utilisateur anonymous	24
	Authentification vs autorisation	25
	Authentification vs autorisation (Suite...)	26
	Que modifier en cas d'évolution ?	27
	Que modifier en cas d'évolution ? (Suite...)	28
	Tests de l'authentification	29
	Résumé pour un nouveau développeur	30

[← Sommaire \(suite...\)](#)[Vue d'ensemble →](#)

Introduction

L'application ToDo & Co utilise le composant Security de Symfony pour gérer l'authentification et les autorisations des utilisateurs.

L'objectif de cette documentation est de permettre à un nouveau développeur de comprendre :

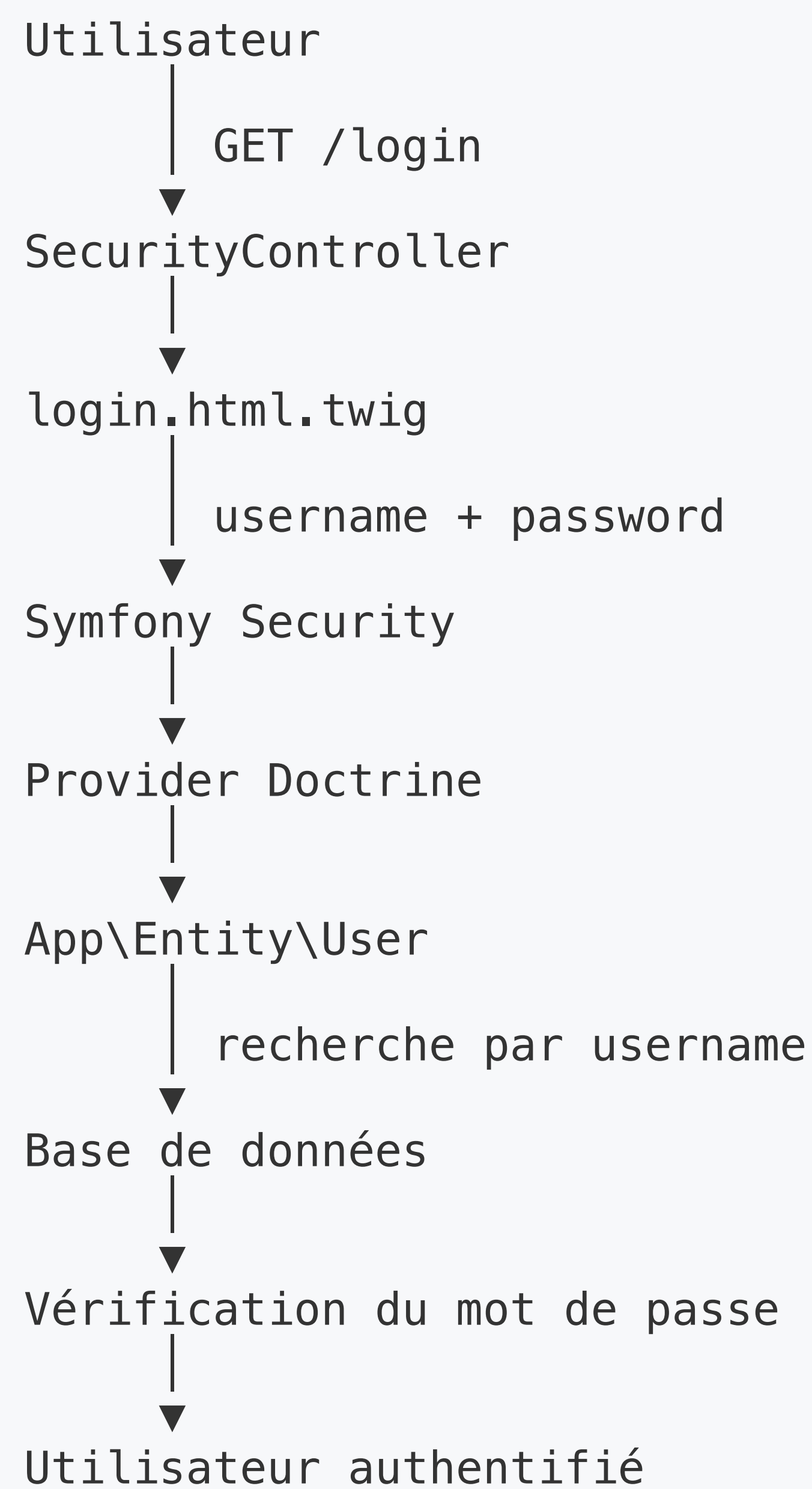
- comment un utilisateur est authentifié ;
- où sont stockés les utilisateurs ;
- comment Symfony retrouve un utilisateur ;
- comment les mots de passe sont protégés ;
- comment les rôles sont gérés ;
- comment les accès aux différentes pages sont contrôlés ;
- quels fichiers interviennent dans le processus d'authentification.

L'authentification repose principalement sur :

```
config/packages/security.yaml
src/entity/user.php
src/controller/securitycontroller.php
templates/security/login.html.twig
```

Vue d'ensemble

Le fonctionnement général de l'authentification est le suivant :



Une fois authentifié, Symfony conserve l'état de l'utilisateur dans la session et utilise ses rôles pour déterminer les fonctionnalités auxquelles il peut accéder.



Fichiers concernés

Les principaux fichiers intervenant dans l'authentification sont :

```
config/
├── packages/
│   └── security.yaml

src/
├── controller/
│   └── securitycontroller.php
├── entity/
│   └── user.php

templates/
├── security/
│   └── login.html.twig
```

Chaque fichier possède un rôle différent.

Fichier	Rôle
security.yaml	Configuration de Symfony Security
User.php	Configuration de Symfony Security
SecurityController.php	Affichage de la page de connexion
login.html.twig	Formulaire de connexion

[← Fichiers concernés](#)[🔑 Hashage des mots de passe →](#)

Configuration de Symfony Security

La configuration se trouve dans :

```
config/packages/security.yaml
```

Elle définit notamment :

- le système de hashage des mots de passe ;
- le fournisseur d'utilisateurs ;
- le firewall ;
- le formulaire de connexion ;
- la déconnexion ;
- les règles d'accès aux différentes routes.

Hashage des mots de passe

La configuration suivante définit le système utilisé pour protéger les mots de passe :

```
password_hashers:  
  app\Entity\User:  
    algorithm: auto
```

Symfony détermine automatiquement l'algorithme de hashage approprié.

Lorsqu'un utilisateur est créé ou que son mot de passe est modifié, le mot de passe ne doit donc pas être enregistré directement en clair dans la base de données.

Dans `UserController`, le hashage est réalisé avec :

```
userpasswordhasherinterface
```

Le principe est :

```
mot de passe saisi  
  ↓  
userpasswordhasherinterface  
  ↓  
mot de passe hashé  
  ↓  
base de données
```

Lors de l'authentification, Symfony compare ensuite le mot de passe fourni avec le hash enregistré.

[← 🔑 Hashage des mots de passe](#)[Entité User →](#)

Fournisseur d'utilisateurs

La configuration suivante définit le système utilisé pour protéger les mots de passe :

```
providers:
  doctrine:
    entity:
      class: App\Entity\User
      property: username
```

Cela signifie que Symfony utilise l'entité :

```
app\Entity\User
```

pour retrouver les utilisateurs.

La recherche est effectuée à partir de la propriété :

```
username
```

Le parcours est donc :

```
username saisi
↓
provider doctrine
↓
app\Entity\User
↓
recherche par username
↓
utilisateur trouvé
```

Le `username` est donc l'identifiant utilisé lors de la connexion.

[← Fournisseur d'utilisateurs](#)[ID Identifiant de l'utilisateur →](#)

Entité User

L'entité utilisateur se trouve dans :

```
src/entity/user.php
```

Elle implémente deux interfaces Symfony :

```
userinterface  
passwordauthenticateduserinterface
```

La déclaration est :

```
class user implements userinterface, passwordauthenticateduserinterface
```

Ces interfaces permettent à l'entité d'être utilisée directement par le composant Security de Symfony.

ID Identifiant de l'utilisateur

Symfony utilise la méthode :

```
getuseridentifiant()
```

pour obtenir l'identifiant de l'utilisateur.

Dans l'application :

```
public function getuseridentifiant(): string
{
    return (string) $this->username;
}
```

L'identifiant retourné est donc le :

```
username
```

Ainsi :

```
username
  ↓
getuseridentifiant()
  ↓
identifiant symfony
```

[← ID](#) Identifiant de l'utilisateur[🔒 Mot de passe →](#)

Stockage des utilisateurs

Les utilisateurs sont stockés dans la base de données via l'entité :

```
app\entity\user
```

La table correspondante est :

```
user
```

Elle est explicitement définie dans l'entité :

```
#[ORM\Table(name: 'user')]
```

Les principales informations stockées sont :

```
user
├── id
├── username
├── password
├── email
└── roles
```

Les rôles sont stockés dans une colonne JSON :

```
#[ORM\Column(type: 'json')]
private $roles = [];
```

Un utilisateur peut donc avoir une valeur telle que :

```
[<< ROLE_USER >>]
```

ou :

```
[<< ROLE_ADMIN >>]
```

[← 📁 Stockage des utilisateurs](#)[🌐 Page de connexion →](#)

Mot de passe

L'entité `User` implémente :

```
passwordauthenticateduserinterface
```

et fournit la méthode :

```
public function getpassword(): string
{
    return $this->password;
}
```

Symfony utilise cette méthode pour récupérer le hash du mot de passe enregistré.

Le mot de passe lui-même n'est pas stocké en clair.

[← 🔒 Mot de passe](#)[🖨️ Formulaire de connexion →](#)

Page de connexion

La page de connexion est gérée par :

```
src/controller/securitycontroller.php
```

La route est :

```
#[route('/login', name: 'login')]
```

La méthode `login()` récupère les informations nécessaires auprès de :

```
AuthenticationUtils
```

Elle transmet au template :

```
[
    'last_username' => $authenticationUtils->getLastUsername(),
    'error' => $authenticationUtils->getLastAuthenticationError(),
]
```

Cela permet notamment d'afficher :

- le dernier nom d'utilisateur saisi ;
- le message d'erreur lorsqu'une authentification échoue.

Formulaire de connexion

Le formulaire est défini dans :

```
templates/security/login.html.twig
```

Il utilise :

```
<form action=« {{ path('login') }} » method=« post »></form>
```

Les champs envoyés à Symfony sont :

```
_username  
_password
```

Le nom d'utilisateur est envoyé avec :

```
<input type=« text » id=« username » name=« _username » />
```

Le formulaire contient également un token CSRF :

```
<input  
  type=« hidden »  
  name=« _csrf_token »  
  value=« {{ csrf_token('authenticate') }} »  
>
```

Ce token correspond à la configuration :

```
enable_csrf: true  
csrf_token_id: authenticate
```

Fonctionnement du formulaire

Lorsqu'un utilisateur valide le formulaire :

```
utilisateur
  ↓
post /login
  ↓
symfony security
  ↓
provider doctrine
  ↓
recherche du username
  ↓
utilisateur trouvé
  ↓
vérification du mot de passe
  ↓
authentification
```

Il est important de noter que `SecurityController` n'effectue pas lui-même la vérification du mot de passe.

Le contrôleur sert principalement à afficher le formulaire et à transmettre les éventuelles informations d'erreur.

La gestion de l'authentification est effectuée par le composant Security de Symfony.

[← ⌂ Fonctionnement du formulaire](#)[Déconnexion →](#)

Firewall

Le firewall principal est configuré dans :

```
config/packages/security.yaml
```

La configuration est :

```
main:
  lazy: true
  pattern: ^/
```

Le firewall `main` couvre donc les routes principales de l'application.

Il utilise le système de formulaire Symfony :

```
form_login:
  login_path: login
  check_path: login
  enable_csrf: true
  csrf_token_id: authenticate
```

La route utilisée pour afficher le formulaire et traiter l'authentification est :

```
/login
```

[← Firewall](#)[Contrôle des accès →](#)

Déconnexion

La déconnexion est également configurée dans le firewall :

```
logout:  
  path: logout  
  target: /
```

Lorsqu'un utilisateur demande :

```
/logout
```

Symfony gère automatiquement la déconnexion.

Après celle-ci, l'utilisateur est redirigé vers :

```
/
```

[← Déconnexion](#)[Accès à /login →](#)

Contrôle des accès

Les règles d'autorisation sont définies avec :

```
access_control:
```

La configuration actuelle est :

```
access_control:
- { path: ^/login, roles: public_access }
- { path: ^/users, roles: role_admin }
- { path: ^/, roles: role_user }
```

Ces règles sont évaluées pour déterminer si l'utilisateur peut accéder à une route.

[← 🚦 Contrôle des accès](#)[👤 Gestion des utilisateurs →](#)

Accès à `/login`

La route :

```
/login
```

utilise :

```
PUBLIC_ACCESS
```

Elle peut donc être consultée sans être authentifié.

C'est nécessaire puisque l'utilisateur doit pouvoir accéder à la page de connexion avant d'être authentifié.

```
utilisateur non authentifié
↓
get /login
↓
public_access
↓
✅ page accessible
```



Gestion des utilisateurs

Les routes commençant par :

```
/users
```

nécessitent :

```
ROLE_ADMIN
```

Ainsi :

```
role_admin
  ↓
/users
  ↓
✅ accès autorisé
```

alors que :

```
role_user
  ↓
/users
  ↓
❌ access denied
```

Cette règle protège notamment les fonctionnalités de gestion des utilisateurs.

[← 🏠 Gestion des utilisateurs](#)[👤 Gestion des rôles →](#)

Accès aux autres pages

La règle :

```
- { path: ^/, roles: role_user }
```

protège toutes les autres routes correspondant au chemin /.

Un utilisateur doit donc être authentifié et disposer du rôle :

```
ROLE_USER
```

pour accéder aux fonctionnalités normales de l'application.

[← !\[\]\(0271f7a0ac0c29ba1f8ceccd44fa7f94_img.jpg\) Accès aux autres pages](#)[↻ Modification du rôle →](#)

Gestion des rôles

L'application utilise deux rôles :

```
ROLE_USER  
ROLE_ADMIN
```

Dans l'entité `User` , la méthode :

```
getRoles()
```

garantit qu'un utilisateur possède toujours au minimum :

```
ROLE_USER
```

Par exemple, si l'utilisateur possède :

```
[ 'ROLE_ADMIN' ]
```

la méthode retourne :

```
ROLE_ADMIN  
ROLE_USER
```

Le rôle administrateur bénéficie donc également des permissions accordées à `ROLE_USER` .

[← 👤 Gestion des rôles](#)[📝 Association avec les tâches →](#)

Modification du rôle

Le rôle peut être défini avec :

```
setRole()
```

La méthode accepte uniquement :

```
ROLE_USER  
ROLE_ADMIN
```

Toute autre valeur provoque une :

```
InvalidArgumentException
```

Cela permet de limiter les rôles disponibles aux deux rôles prévus par l'application.

[← Modification du rôle](#)[Autorisation des actions sur les tâches →](#)

Association avec les tâches

L'utilisateur possède également une relation avec les tâches :

```
#[ORM\OneToMany(mappedBy: 'user', targetEntity: Task::class)]  
private Collection $tasks;
```

La relation est donc :



Lorsqu'une tâche est ajoutée à un utilisateur avec :

```
$user->addTask($task);
```

l'utilisateur est automatiquement défini comme propriétaire :

```
$task->setUser($this);
```

Cela permet de maintenir la cohérence de l'association entre les deux entités.



Autorisation des actions sur les tâches

L'authentification permet de connaître l'utilisateur connecté.

L'autorisation détermine ensuite ce que cet utilisateur peut faire.

Pour les tâches, les décisions sont prises notamment par :

```
src/Security/Voter/TaskVoter.php
```

Le Voter prend en charge :

```
EDIT  
TOGGLE  
DELETE
```

Le principe est :

```
Utilisateur authentifié  
↓  
Action sur une Task  
↓  
TaskVoter  
↓  
Vérification des droits  
↓  
GRANTED / DENIED
```

Le propriétaire peut notamment agir sur sa propre tâche.

Un autre utilisateur ne peut pas modifier ou supprimer une tâche qui ne lui appartient pas.

Les tâches associées à :

```
anonymous
```

possèdent une règle particulière : leur suppression est réservée aux administrateurs.

[← 🛡️ Autorisation des actions sur les tâches](#)[🔑 Authentification vs autorisation →](#)

Utilisateur **anonymous**

Les anciennes tâches créées avant l'association obligatoire à un utilisateur sont rattachées à un utilisateur spécifique :

anonymous

Cet utilisateur possède un compte dédié dans la table user.

Son rôle permet de conserver les anciennes tâches tout en maintenant désormais une association obligatoire :

```
Task
  ↓
user_id
  ↓
anonymous
```

Les utilisateurs standards ne peuvent pas supprimer ces tâches.

Les administrateurs peuvent les supprimer conformément aux règles définies par le **TaskVoter** .



Authentification vs autorisation

Il est important de distinguer les deux notions.

- Authentification
- Autorisation

Elle répond à la question :

« Qui est l'utilisateur ? »

Elle est principalement gérée par :

Symfony Security
Provider Doctrine
User
Firewall

Suite...

Il est important de distinguer les deux notions.

Authentification

Autorisation

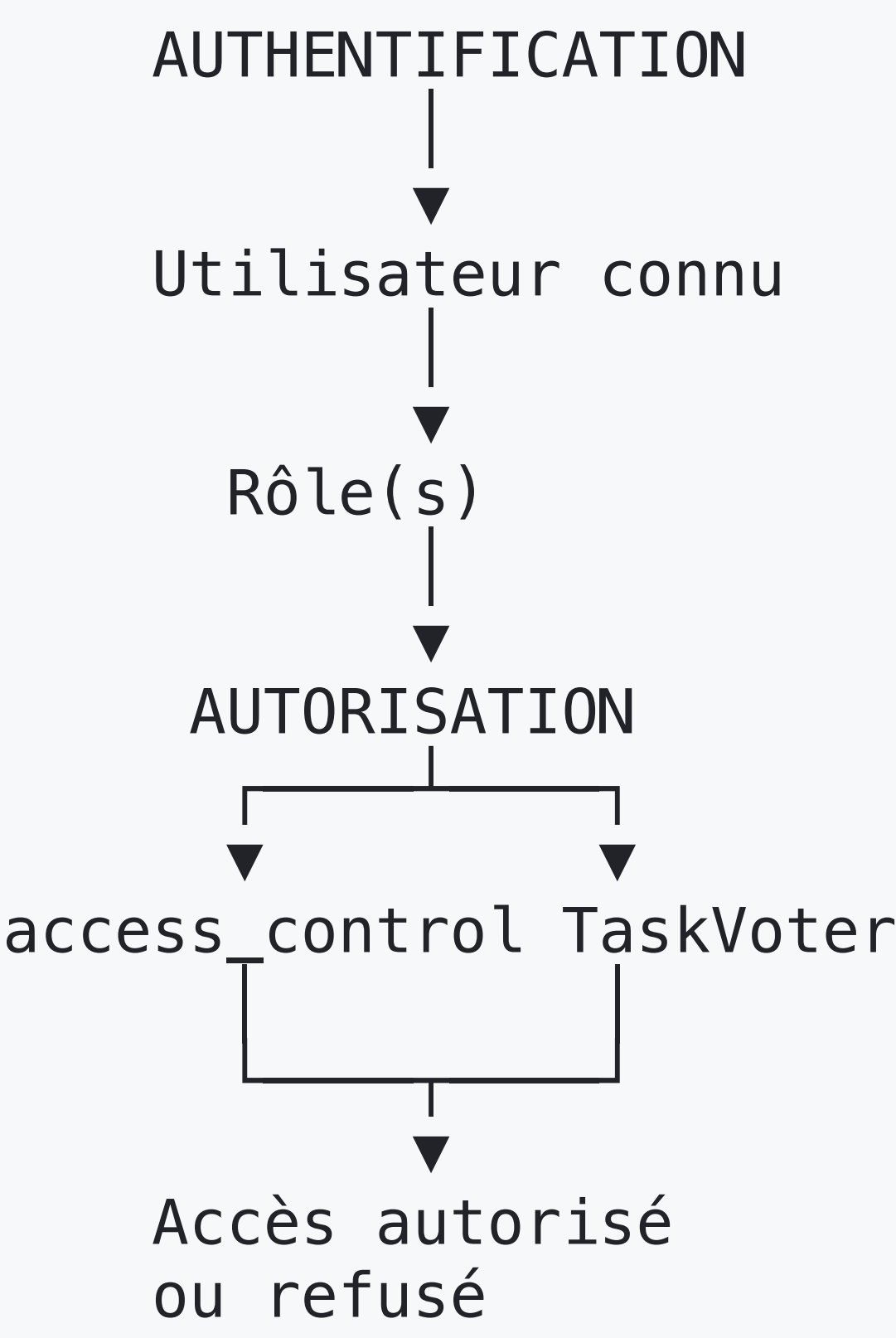
Elle répond à la question :

« Cet utilisateur a-t-il le droit d'effectuer cette action ? »

Elle repose notamment sur :

access_control
rôles
TaskVoter

Le fonctionnement global peut être résumé ainsi :



[← ↻ Modification du rôle](#)[Suite... →](#)

Que modifier en cas d'évolution ?

Modifier le comportement de connexion

Consulter principalement :

```
config/packages/security.yaml  
src/controller/securitycontroller.php  
templates/security/login.html.twig
```

Modifier les informations d'un utilisateur

Consulter :

```
src/entity/user.php  
src/form/usertype.php  
src/controller/usercontroller.php
```

Modifier les rôles disponibles

Consulter :

```
src/entity/user.php  
src/form/usertype.php  
config/packages/security.yaml
```

et vérifier également les tests associés.

Modifier les autorisations sur les tâches

Consulter :

```
src/Security/Voter/TaskVoter.php
```

et les tests :

```
tests/Security/Voter/TaskVoterTest.php
```

[← !\[\]\(144bd6ca55a232a7ce6ff9d490e451c8_img.jpg\) Que modifier en cas d'évolution ?](#)[Tests de l'authentification → !\[\]\(909ff976bf47376a7a813dc1902dc014_img.jpg\)](#)

Suite...

Modifier les accès aux routes

Consulter :

```
config/packages/security.yaml
```

Les modifications de sécurité doivent toujours être accompagnées de tests permettant de vérifier les accès autorisés et refusés.

Tests de l'authentification

L'authentification et les autorisations sont couvertes par les tests fonctionnels.

Les tests permettent notamment de vérifier :

- l'accès à `/login` ;
- la connexion avec un utilisateur valide ;
- le refus d'identifiants invalides ;
- la redirection d'un utilisateur non authentifié ;
- l'accès d'un `ROLE_USER` aux fonctionnalités normales ;
- l'accès d'un `ROLE_ADMIN` à la gestion des utilisateurs ;
- le refus d'accès d'un `ROLE_USER` à `/users`.

Les tests du `TaskVoter` vérifient quant à eux les autorisations spécifiques aux tâches.

Les tests sont exécutés avec :

```
vendor/bin/phpunit
```

[← Tests de l'authentification](#)

Résumé pour un nouveau développeur

Pour comprendre rapidement l'authentification du projet, retenir les éléments suivants :

1. `user.php`
↓
représente l'utilisateur symfony
2. `security.yaml`
↓
configure security, le provider, le firewall et les accès
3. `securitycontroller.php`
↓
affiche la page de connexion
4. `login.html.twig`
↓
contient le formulaire de connexion
5. `userpasswordhasherinterface`
↓
protège les mots de passe
6. `access_control`
↓
protège les routes selon les rôles
7. `taskvoter`
↓
protège les actions sur les tâches

Le principe général est donc :

```
utilisateur  
  ↓  
authentification symfony  
  ↓  
user chargé depuis doctrine  
  ↓  
rôles récupérés  
  ↓  
contrôle des accès  
  ↓  
action autorisée ou refusée
```

Cette architecture permet de centraliser les règles de sécurité tout en séparant clairement l'authentification, la gestion des rôles et les autorisations métier.